PFE Execution Guide: Classical Continuous-Time Stochastic Optimal Control (Fully Observed)

Hamiltonian-Partials Reformulation + Pathwise Discrete Dual Gradients

Student: Youssef Kallala, youssef.kallala@ept.ucar.tn
Supervisors: R. Tempone & Erik von Schwerin

February 2026 – [TODO: End date]

Abstract

This guide is a step-by-step playbook to execute a Bachelor/PFE project on continuous-time stochastic optimal control with fully observed state. The numerical backbone is a *Hamiltonian-partials reformulation* that turns policy search into an unconstrained optimization over a smooth potential function $\bar{u}(t, x)$, combined with simulation-based evaluation and *pathwise discrete dual* gradients. We begin with analytic LQR/LQG benchmarks as unit tests, then scale to nonlinear and non-smooth (switching) Hamiltonians, and finish with a reproducible codebase, experiments, and thesis-ready figures/tables.

Contents

1 Administrative snapshot (fill once) **3**

2 Scope control (keep it finishable) **3**
2.1 In-scope . . . 3
2.2 Out-of-scope . . . 4
2.3 Non-negotiable deliverables . . . 4

3 Problem statement (precise mathematical formulation) **4**
3.1 Controlled diffusion (fully observed) . . . 4

4 Core theoretical objects (freeze notation here) **4**
4.1 Notation (freeze once and keep consistent) . . . 4
4.2 Extended Hamiltonian and HJB . . . 5
4.3 Hamiltonian-partials reformulation over a potential . . . 5
4.4 On-policy error identity (diagnostic) . . . 6

5 Warm-up examples (in increasing difficulty) **6**
5.1 Example 1 (easiest): deterministic LQR (unit test 1) . . . 6
5.2 Example 2: stochastic LQG with additive noise (unit test 2) . . . 6
5.3 Example 3: 2D stochastic LQ benchmark (unit test 3; matches main reference) . . . 7
5.4 Example 4: non-smooth switching Hamiltonian (high-dimensional stress test) . . . 7

6 Core algorithmic workflow (what you implement) **8**
6.1 Smoothing H to H_δ (why, and what it changes) . . . 8
6.2 Discretization . . . 8
6.3 Pathwise gradient via discrete dual (avoid backward SDEs) . . . 9
6.4 High-level pseudocode . . . 10

7 Numerical experiments plan (metrics, plots, baselines) **10**
7.1 Metrics . . . 10
7.2 Experiment ladder (easy → hard) . . . 10
7.3 What counts as "success"? . . . 10

8 Reproducible software plan (repo layout, modules, tests) **11**
8.1 Repository layout (suggested, adapt as you like) . . . 11
8.2 Unit tests (minimum set) . . . 11
8.3 Reproducibility checklist . . . 12

9 Risks and failure modes (with early-warning signs) **12**

10 Week-by-week timeline (16 weeks; deliverables and checkpoints) **12**

11 References (minimal) **13**

A Appendix: quick checklist for every new model **14**

How to use this guide (read now vs. later)

What is a PFE? At KAUST (and in many French-speaking systems), *PFE* stands for *Projet de fin d'études*: a capstone Bachelor project whose deliverables are (i) a reproducible codebase, (ii) a short thesis/report, and (iii) a presentation.

Scope control without hiding the roadmap. To avoid overwhelming the student while keeping the end-to-end plan visible, this document is written as a single "master" guide.

- **Weeks 1–4 (read now):** "Administrative snapshot" through "Warm-up examples" (setup, problem, notation, benchmarks).
- **Weeks 5–12 (unlock gradually):** "Core algorithmic workflow" and "Numerical experiments plan" (algorithm, experiments).
- **Weeks 13–16 (roadmap for writing/packaging):** "Reproducible software plan" through "Week-by-week timeline".

1 Administrative snapshot (fill once)

- **Code stack (choose one)**: Python (recommended) / MATLAB.
- **Repository**: [TODO: Repository URL (Bitbucket / GitHub / KAUST GitLab)] (pick the platform that makes collaborator access easiest)
- **Reproduce key figures/tables**:
  `python -m scripts.run_experiment --config configs/lqg_1d.yaml` (optionally provide `python -m scripts.run_suite` for tiny runs)
- **Primary references**:
  1. Main reference: *Controlled Diffusions as Unconstrained Smooth Optimization in a Potential* (Tempone, working draft).
  2. One SOC textbook: Fleming–Soner *Controlled Markov Processes and Viscosity Solutions.*
  3. One numerical SOC reference: Kushner–Dupuis *Numerical Methods for Stochastic Control Problems in Continuous Time.*

2 Scope control (keep it finishable)

2.1 In-scope

- Fully observed controlled Itô diffusions in continuous time, finite horizon $[0, T]$.
- Markov (feedback) controls $\alpha_t = \alpha(t, X_t)$ (closed-loop).
- Simulation-based optimization over a parametric potential $\bar{u}_\theta(t, x)$ (splines / smooth NN / RFF).
- Benchmarks with known solutions (LQR/LQG) and at least one challenging example (nonlinear or non-smooth Hamiltonian).
- Reproducible implementation with unit tests and experiment scripts.

2.2 Out-of-scope

- Partial observation / filtering (unless added as optional extension).
- High-dimensional HJB PDE solvers on grids (curse of dimensionality).
- Proving new theorems beyond reproducing what is needed for implementation.

2.3 Non-negotiable deliverables

- A clean code repository with: (i) unit tests based on analytic benchmarks, (ii) scripts that regenerate all figures/tables.
- An experimental report: cost gaps, convergence plots, robustness sweeps.
- A thesis PDF that mirrors your repo structure (problem → methods → experiments).

3 Problem statement (precise mathematical formulation)

3.1 Controlled diffusion (fully observed)

Fix a horizon $T > 0$, state dimension d, Brownian dimension r, and a control set $\mathcal{A}$. Let $(W_t)_{t\in[0,T]}$ be an r-dimensional Brownian motion on a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t), \mathbb{P})$. For a (Markov) feedback control $\alpha(t, x) \in \mathcal{A}$, define the controlled SDE

$$\mathrm{d}X_t = a(X_t, \alpha(t, X_t))\,\mathrm{d}t + \sigma(X_t, \alpha(t, X_t))\,\mathrm{d}W_t, \qquad X_0 = y \in \mathbb{R}^d, \tag{1}$$

where $a : \mathbb{R}^d \times \mathcal{A} \to \mathbb{R}^d$ and $\sigma : \mathbb{R}^d \times \mathcal{A} \to \mathbb{R}^{d\times r}$ satisfy conditions ensuring well-posedness.

The objective is to minimize the expected cost

$$J(\alpha) := \mathbb{E}\left[\int_0^T h(X_t, \alpha(t, X_t))\,\mathrm{d}t + g(X_T)\right], \tag{2}$$

where $h : \mathbb{R}^d \times \mathcal{A} \to \mathbb{R}$ is running cost and $g : \mathbb{R}^d \to \mathbb{R}$ is terminal cost.

Numerical target (what you must compute). Your practical target is **a near-optimal feedback policy** (and its cost) for one or more chosen problem instances $(T, y, \mathcal{A}, a, \sigma, h, g)$, plus quantitative diagnostics:

- **Primary:** $\alpha^\star_\theta(t, x)$ induced by $\bar{u}_\theta(t, x)$ (see below) and an estimate of its achieved cost.
- **Secondary:** convergence/stability curves vs discretization Δt and Monte Carlo budget M.

Reference/benchmark objects (not your main computation). For special cases (LQR/LQG), the value function and optimal policy are known in closed form via a Riccati equation; these provide unit tests.

4 Core theoretical objects (freeze notation here)

4.1 Notation (freeze once and keep consistent)

Throughout the project:

- Time horizon $T > 0$; state dimension d; Brownian dimension r; control dimension m (if controls are vectors).

- $\mathbb{S}^d$ is the space of real symmetric $d \times d$ matrices; $\mathbb{S}^d_+$ are symmetric positive semidefinite (PSD) matrices.
- Control set $\mathcal{A}$ is the set of admissible control values (e.g. a convex subset of $\mathbb{R}^m$ or a finite set).
- An *admissible control* is a progressively measurable process $\alpha = (\alpha_t)_{t\in[0,T]}$ with values in $\mathcal{A}$ such that the controlled SDE is well-posed and the cost is finite.

We assume the usual Lipschitz/linear-growth conditions on (a, σ) so the SDE is well posed, and we only differentiate objects when the needed smoothness holds (or after smoothing; see Section 6.1).

4.2 Extended Hamiltonian and HJB

For $(x, \lambda, V, \gamma) \in \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{S}^d \times \mathbb{R}$ define the *extended Hamiltonian*

$$H(x, \lambda, V, \gamma) := \inf_{\alpha\in\mathcal{A}} \left\{ \langle \lambda, a(x, \alpha)\rangle + \frac{1}{2}\mathrm{Tr}\big((\sigma\sigma^\top)(x, \alpha)\, V\big) + \gamma\, h(x, \alpha) \right\}. \tag{3}$$

Define the *value function* (optimal cost-to-go) by

$$U(t, x) := \inf_{\alpha \text{ admissible}} \mathbb{E}\left[\int_t^T h(X_s^{t,x,\alpha}, \alpha_s)\, \mathrm{d}s + g(X_T^{t,x,\alpha}) \right], \tag{4}$$

where $X^{t,x,\alpha}$ solves the controlled SDE starting from $X_t = x$ under the control α. When U is smooth enough (classical solution), it satisfies the Hamilton–Jacobi–Bellman (HJB) PDE

$$\partial_t U(t, x) + H\big(x, \nabla U(t, x), \nabla^2 U(t, x), 1\big) = 0, \qquad U(T, x) = g(x). \tag{5}$$

When U is smooth enough, it induces an optimal feedback by the minimizer in (3).

4.3 Hamiltonian-partials reformulation over a potential

Instead of solving (5) on a grid, we optimize over a *potential* $\bar{u}(t, x)$, whose derivatives play the roles of costate and Hessian:

$$\lambda(t, x) := \nabla_x \bar{u}(t, x), \qquad V(t, x) := \nabla_x^2 \bar{u}(t, x) \in \mathbb{S}^d. \tag{6}$$

Define the closed-loop dynamics driven by Hamiltonian partial derivatives:

$$\mathrm{d}X_t = \partial_\lambda H\big(X_t, \nabla\bar{u}(t, X_t), \nabla^2\bar{u}(t, X_t), 1\big)\, \mathrm{d}t + \Sigma_t\, \mathrm{d}W_t, \qquad \Sigma_t\Sigma_t^\top = 2\, \partial_V H\big(X_t, \nabla\bar{u}, \nabla^2\bar{u}, 1\big), \tag{7}$$

and the induced objective

$$J(\bar{u}) = \mathbb{E}\left[\int_0^T \partial_\gamma H\big(X_t, \nabla\bar{u}(t, X_t), \nabla^2\bar{u}(t, X_t), 1\big)\, \mathrm{d}t + g(X_T) \right]. \tag{8}$$

The project method is to minimize $J(\bar{u}_\theta)$ over parameters θ (splines/NN/etc.).

Why this is attractive. If H is concave in (λ, V, γ) and smooth enough, then (i) the diffusion metric $\partial_V H$ is positive semidefinite (so Σ exists), (ii) one can compute gradients of the discretized objective without backward SDEs (discrete dual), and (iii) there are *a posteriori* error identities you can use as diagnostics.

4.4 On-policy error identity (diagnostic)

Let U be the (unknown) exact HJB solution and let $\bar{u}$ be any candidate potential. For the on-policy trajectory X_t under (7), there is an identity of the form

$$J(\bar{u}) - U(0, y) = \mathbb{E}\left[\int_0^T \mathcal{E}(t, X_t)\,\mathrm{d}t\right], \tag{9}$$

where $\mathcal{E}(t, X_t) \geq 0$ under concavity assumptions. This gives a conceptual *nonnegativity* check: your learned policy should not appear "better than optimal" in any diagnostic that is provably nonnegative.

Mini-checks (do these every time you change code).

- Dimensional analysis: $\lambda \in \mathbb{R}^d$, $V \in \mathbb{S}^d$, $\partial_V H \in \mathbb{S}^d_+$.
- Symmetry: enforce $V = \frac{1}{2}(V + V^\top)$ numerically.
- PSD: clip negative eigenvalues of $\partial_V H$ if needed (and log how often you clipped).

5 Warm-up examples (in increasing difficulty)

5.1 Example 1 (easiest): deterministic LQR (unit test 1)

Model. Let $X_t \in \mathbb{R}^d$ follow the ODE

$$\dot{X}_t = AX_t + Bu_t, \qquad X_0 = y,$$

and minimize

$$J(u) = \int_0^T \left(X_t^\top Q X_t + u_t^\top R u_t\right)\,\mathrm{d}t + X_T^\top S X_T,$$

with $Q, S \succeq 0$ and $R \succ 0$.

Reference solution (analytic). The value function is quadratic $U(t, x) = x^\top P(t) x$ where P solves the Riccati ODE

$$-\dot{P} = A^\top P + PA - PBR^{-1}B^\top P + Q, \qquad P(T) = S,$$

and the optimal feedback is $u^\star(t, x) = -R^{-1}B^\top P(t)x$.

What you solve numerically. Integrate the Riccati ODE backward in time; simulate closed-loop trajectories; check that the cost matches $y^\top P(0) y$.

Sanity checks. $P(t)$ remains symmetric; if $Q, S \succeq 0$ then $P(t) \succeq 0$ (numerically: eigenvalues nonnegative up to tolerance).

5.2 Example 2: stochastic LQG with additive noise (unit test 2)

Model. Let

$$\mathrm{d}X_t = (AX_t + Bu_t)\,\mathrm{d}t + \Sigma\,\mathrm{d}W_t,$$

with the same quadratic cost as Example 1.

Reference. The optimal feedback remains linear with the same Riccati equation for $P(t)$. (There may be an additional scalar term in the value function due to noise; for unit tests, compare *control law* and Monte Carlo costs.)

What you solve numerically.

- Implement Euler–Maruyama for the closed-loop SDE under $u^\star(t,x)$.
- Estimate $J(u^\star)$ by Monte Carlo and verify convergence as $\Delta t \downarrow 0$ and $M \uparrow \infty$.

5.3 Example 3: 2D stochastic LQ benchmark (unit test 3; matches main reference)

This is the explicit benchmark described in the main reference.

Model and cost. In $\mathbb{R}^2$, consider

$$\mathrm{d}X_t = AX_t\,\mathrm{d}t + Bu_t\,\mathrm{d}t + \sigma\,\mathrm{d}W_t,$$

with

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \sigma = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}, \quad Q = I_2,\ R = 1,\ S = I_2,\ T = 1.$$

The cost is

$$J(u) = \mathbb{E}\left[\int_0^T \left(X_t^\top Q X_t + u_t^\top R u_t\right)\,\mathrm{d}t + X_T^\top S X_T\right].$$

Reference solution (analytic). $U(t,x) = x^\top P(t) x$ with Riccati

$$-\dot{P} = A^\top P + PA - PBR^{-1}B^\top P + Q, \qquad P(T) = S,$$

and $u^\star(t,x) = -R^{-1}B^\top P(t)x$.

Hamiltonian (write it explicitly). With costate $\lambda \in \mathbb{R}^2$ and Hessian $V \in \mathbb{R}^{2\times 2}$,

$$H(x,\lambda,V,1) = \min_{u\in\mathbb{R}} \left\{\lambda^\top(Ax+Bu) + \frac{1}{2}\mathrm{Tr}(\sigma\sigma^\top V) + x^\top Q x + u^\top R u\right\}.$$

The minimizer is $u^\star(x,\lambda) = -\frac{1}{2}R^{-1}B^\top\lambda$.

What you solve numerically (this is where your project starts). Implement the Hamiltonian-partials pipeline:

- Implement H, $\partial_\lambda H$, $\partial_V H$, $\partial_\gamma H$ for this LQ case (closed forms).
- Pick a parametric potential $\bar{u}_\theta(t,x)$ and run the *optimization-over-potential* algorithm. Verify you recover the quadratic U.

Acceptance tests.

- Your learned $\bar{u}_\theta$ should converge (up to approximation error) to $x^\top P(t)x$.
- Your learned policy cost should match the analytic optimal cost within a tolerance you set (e.g., $< 1\%$ at sufficiently small Δt).

5.4 Example 4: non-smooth switching Hamiltonian (high-dimensional stress test)

Use the "freeze/continue" basket example to test (i) non-smooth Hamiltonians and (ii) smoothing.

Model. In $\mathbb{R}^d_+$, let $\alpha_t \in \{0,1\}$ and

$$\mathrm{d}X_t = \alpha_t\big(a(X_t)\,\mathrm{d}t + b(X_t)\,\mathrm{d}W_t\big),$$

so that $\alpha_t = 0$ freezes the state. A concrete choice is $a(x) = 0$ and $b(x) = \mathrm{diag}(x)\sigma$ (lognormal basket). The terminal payoff is a basket put

$$g(x) = \left(K - \sum_{i=1}^{d} w_i x_i\right)_+, \qquad \sum_{i=1}^{d} w_i = 1,$$

and $h \equiv 0$.

Hamiltonian and smoothing (key concept). The Hamiltonian is a pointwise minimum of two affine branches, hence concave but non-smooth. A soft-min regularization (log-sum-exp) yields a smooth H_δ; use it to define $\partial_\lambda H_\delta$ and $\partial_V H_\delta$ and to simulate stable dynamics.

What you solve numerically. Compare three policies:

- Baseline A: always continue ($\alpha_t \equiv 1$).
- Baseline B: always freeze immediately ($\alpha_t \equiv 0$).
- Learned policy from $\bar{u}_\theta$ using smoothed Hamiltonian H_δ.

This gives a clean, scalable benchmark even when no closed-form optimum is available.

6 Core algorithmic workflow (what you implement)

6.1 Smoothing H to H_δ (why, and what it changes)

The extended Hamiltonian $H(x, \lambda, V, \gamma)$ is defined as an infimum over controls; even when a, σ, h are smooth, the map $(\lambda, V) \mapsto H$ can be *non-smooth* if the minimizer $\alpha^\star$ is not unique or switches abruptly (e.g. bounded controls, bang–bang regimes, discrete controls). This matters because our optimization uses gradients with respect to the potential parameters θ.

A standard fix is to replace H by a smooth, concave surrogate H_δ (e.g. log-sum-exp / soft-min of a finite bundle of affine minorants). Key properties we want from H_δ:

- $H_\delta \approx H$ (bias controlled by δ).
- H_δ is C^1 (often C^2) in (λ, V) so that $\partial_\lambda H_\delta$, $\partial_V H_\delta$, $\partial_\gamma H_\delta$ exist and are Lipschitz.
- $\partial_V H_\delta(x, \lambda, V, \gamma) \in \mathbb{S}^d_+$ so the induced diffusion matrix is well-defined.

If your control set and problem data already make H smooth with a unique minimizer, you may skip smoothing; otherwise, use H_δ for stable gradients and simulations (and record δ in every experiment).

6.2 Discretization

Use Euler–Maruyama on a grid $t_n = n\Delta t$, $n = 0, \ldots, N$ with $\Delta t = T/N$. For a parametric potential $\bar{u}_\theta(t, x)$ define

$$\tilde{a}_\theta(t,x) := \partial_\lambda H_\delta\big(x, \nabla\bar{u}_\theta(t,x), \nabla^2\bar{u}_\theta(t,x), 1\big),$$

$$\tilde{Q}_\theta(t,x) := 2\,\partial_V H_\delta\big(x, \nabla\bar{u}_\theta(t,x), \nabla^2\bar{u}_\theta(t,x), 1\big), \qquad \Sigma_\theta\Sigma_\theta^\top = \tilde{Q}_\theta,$$

$$\tilde{h}_\theta(t,x) := \partial_\gamma H_\delta\big(x, \nabla\bar{u}_\theta(t,x), \nabla^2\bar{u}_\theta(t,x), 1\big).$$

Interpretation of the "tilded" quantities. The triple $(\tilde{a}_\theta, \tilde{Q}_\theta, \tilde{h}_\theta)$ are *the drift, diffusion metric, and running cost under the feedback induced by* $\bar{u}_\theta$. Indeed, if the infimum defining H_δ is attained by some (possibly δ-regularized) minimizer $\alpha^\star_\delta(x, \lambda, V, \gamma)$, then envelope/Danskin identities give

$$\partial_\lambda H_\delta = a\big(x, \alpha^\star_\delta\big), \qquad 2\,\partial_V H_\delta = (\sigma\sigma^\top)\big(x, \alpha^\star_\delta\big), \qquad \partial_\gamma H_\delta = h\big(x, \alpha^\star_\delta\big).$$

So $\tilde{h}_\theta$ is not a new cost function: it is simply the original running cost evaluated at the induced (approximate) optimal control.

Why Σ_θ exists. By construction $\tilde{Q}_\theta(t, x) \in \mathbb{S}^d_+$, hence there exists a matrix square root $\Sigma_\theta(t, x)$ such that $\Sigma_\theta \Sigma_\theta^\top = \tilde{Q}_\theta$ (for example the symmetric PSD square root). Numerically, compute it via an eigenvalue decomposition (with small diagonal "jitter" if needed) to avoid failures when $\tilde{Q}_\theta$ is nearly singular.

Then simulate

$$X_{n+1} = X_n + \tilde{a}_\theta(t_n, X_n)\Delta t + \Sigma_\theta(t_n, X_n)\sqrt{\Delta t}\, Z_n, \qquad Z_n \sim \mathcal{N}(0, I_r).$$

Estimate the discrete objective by Monte Carlo with M trajectories:

$$\hat{J}_h(\theta) = \frac{1}{M} \sum_{m=1}^{M} \left(\sum_{n=0}^{N-1} \tilde{h}_\theta(t_n, X_n^{(m)})\Delta t + g(X_N^{(m)}) \right).$$

6.3 Pathwise gradient via discrete dual (avoid backward SDEs)

To optimize θ, you need $\nabla_\theta \hat{J}_h(\theta)$. Use the discrete adjoint (dual) recursion:

- Forward: store (X_n, Z_n) along each trajectory.
- Backward: compute an adjoint sequence P_n (dimension d) backward in n.
- Accumulate the gradient using the formula in the main reference (terms involving $\partial_\theta \tilde{h}_\theta$, $\partial_\theta \tilde{a}_\theta$, and $\partial_\theta \Sigma_\theta$).

Practically, implement this with automatic differentiation (JAX/PyTorch) but be careful with the matrix square root in Σ_θ: use a PSD-safe factorization (eigen or Cholesky + jitter) and log any eigenvalue clipping.

6.4 High-level pseudocode

Algorithm 1 Simulation-based optimization over the potential $\bar{u}_\theta$

Input: model (a, σ, h, g), horizon T, steps N, MC batch size M, smoothing δ, initial θ_0
for $k = 0, 1, 2, \ldots$ **do**
  Simulate M trajectories $\{X_n^{(m)}\}_{n=0}^N$ under current θ_k (Euler–Maruyama)
  Compute objective estimate $\hat{J}_h(\theta_k)$
  Compute gradient estimate $\widehat{\nabla_\theta J_h}(\theta_k)$ using discrete dual adjoint
  Update $\theta_{k+1} \leftarrow \theta_k - \eta_k \widehat{\nabla_\theta J_h}(\theta_k)$ (SGD/Adam/SVRG)
  Diagnostics: cost curve, gradient norm, PSD-clipping rate, stability/constraint violations
  if stagnation and under-parameterization suspected **then**
    Enrich $\bar{u}_\theta$ representation (e.g., add spline knots / widen NN) and continue
  end if
end for
Output: learned potential $\bar{u}_{\theta^\star}$ and induced feedback policy

7 Numerical experiments plan (metrics, plots, baselines)

7.1 Metrics

- **Cost** $\hat{J}_h(\theta)$ with standard error / confidence intervals from R independent replicates. *Practice:* run R independent runs by advancing the PRNG stream (avoid reseeding inside a run); optionally store the Brownian increments to get bit-level reproducibility.
- **Cost gap** vs analytic optimum on LQR/LQG: $\frac{\hat{J}_h(\theta) - J^\star}{J^\star}$.
- **Discretization convergence**: plot cost vs Δt and check stabilization.
- **Monte Carlo variance**: plot standard error vs M.
- **Stability diagnostics**: frequency of PSD clipping; explosion rates; constraint violations.

7.2 Experiment ladder (easy → hard)

1. **Exp 0 (sanity):** deterministic LQR; verify Riccati solver and closed-loop simulator.
2. **Exp 1 (benchmark):** stochastic LQG; verify policy and Monte Carlo convergence.
3. **Exp 2 (main method validation):** optimize over $\bar{u}_\theta$ on the 2D LQ case; show recovery of quadratic value and cost.
4. **Exp 3 (robustness sweep):** vary $(\Delta t, M)$ and optimizer hyperparameters; report stability.
5. **Exp 4 (non-smooth extension):** switching/binary Hamiltonian with smoothing; compare to baselines always-continue / always-freeze.

7.3 What counts as "success"?

For analytic benchmarks, target a small cost gap (e.g. $< 1\%$) at sufficiently small Δt and adequate M. For non-analytic cases, require consistent improvement over baselines plus stable training (no explosions, low clipping rate).

8 Reproducible software plan (repo layout, modules, tests)

8.1 Repository layout (suggested, adapt as you like)

The exact folder names do *not* matter. What matters is that: (i) core code is reusable (not copied between notebooks), (ii) there are `tests/` that cover the analytic benchmarks, and (iii) there are small scripts that regenerate the figures/tables used in the thesis.

Platform choice. Use Bitbucket, GitHub, or KAUST GitLab — whichever makes collaborator access easiest. A minimal repo on day 1 (README + empty folders) is enough; refine structure later.

A minimal, realistic structure for a Bachelor project.

```
repo/
  README.md
  (requirements.txt | environment.yml) # pip OR conda; pick one and document it
  soc/ # your Python package (name is arbitrary)
    __init__.py
    models.py # a(x,u), sigma(x,u), h(x,u), g(x)
    hamiltonian.py # H or H_delta and partials
    potential.py # u_theta, grad, Hessian (autodiff wrapper)
    simulate.py # Euler-Maruyama, Brownian increments
    dual.py # discrete adjoint / dual recursion
    optimize.py # SGD/Adam loop, logging
    repro.py # RNG handling, saving/loading run metadata
  scripts/
    run_experiment.py # run ONE experiment and save figures/results
    run_suite.py # optional: run a small list of experiments
  tests/
    test_lqr_riccati.py
    test_lqg_cost_matches_value.py
    test_hamiltonian_partials_lq.py
    test_dual_gradient_fd_check.py
  figures/ # auto-generated (do not edit by hand)
  results/ # saved metrics, checkpoints, RNG state
```

On configuration files (optional). If you like, store experiment parameters in `configs/` (YAML/JSON/TOML). This is only to make runs reproducible and comparable; a simple Python dictionary is fine. If you use `configs/`, then `run_experiment.py` should load one config, while `run_suite.py` just loops over a few configs for convenience (only when computations are small).

Environment files (what are they?). `requirements.txt` is the list of Python packages for `pip`; `environment.yml` is the equivalent for `conda`. Pick one approach, document installation in the README, and record package versions in every saved run.

8.2 Unit tests (minimum set)

- **Riccati symmetry test:** $P(t) = P(t)^\top$ within tolerance.
- **LQR value test:** Monte Carlo (with $\Sigma = 0$) matches $y^\top P(0) y$.
- **LQ Hamiltonian partials:** compare analytic $\partial_\lambda H$, $\partial_V H$, $\partial_\gamma H$ against your code outputs.
- **Gradient check:** for small dimension, compare the discrete-dual gradient with finite differences on θ (central difference). A practical step size is $\varepsilon = 10^{-4}(1 + |\theta_i|)$ in float64; decrease/increase if you see cancellation/noise.

8.3 Reproducibility checklist

- Randomness: log library versions and RNG state. A fixed seed is helpful but not a guarantee across machines/libraries; for strict reproducibility, store the generated Brownian increments (or the full RNG state) for the final reported runs.
- Store configs and git commit hash with results.
- Use common random numbers (same Brownian increments) when comparing policies.
- Save intermediate checkpoints of θ and training curves.

9 Risks and failure modes (with early-warning signs)

1. **PSD violations in $\partial_V H$ (diffusion metric).**
   Symptom: Cholesky fails / NaNs.
   Mitigation: symmetrize and project to PSD with small diagonal jitter; consider smoothing H_δ.
2. **Gradient explosion / training instability.**
   Symptom: objective diverges, huge gradients, frequent clipping.
   Mitigation: smaller step size, gradient clipping, variance reduction, increase M, smoother $\bar{u}_\theta$.
3. **Under-parameterized potential.**
   Symptom: objective stagnates far above benchmark; cannot match quadratic in LQ tests.
   Mitigation: enrich basis/NN; monitor $\|\nabla^2 \bar{u}_\theta\|$ variability.
4. **Discretization bias mistaken for learning progress.**
   Symptom: policy looks better at coarse Δt but worsens when refining.
   Mitigation: always run a Δt-refinement check for final results.
5. **Silent bugs in derivatives.**
   Symptom: training "works" but fails gradient check on tiny problem.
   Mitigation: implement finite-difference checks early and keep them in CI.

10 Week-by-week timeline (16 weeks; deliverables and checkpoints)

Guiding rule. Commits should happen on the scale of hours (whenever something works or breaks). Each week ends with: (i) a *push* to the remote repo, (ii) a short weekly update (what worked, what failed), and (iii) at least one new plot or one passing test.

1. **Week 1:** Set up repo + environment; implement deterministic LQR simulator and Riccati solver; write notation page.
2. **Week 2:** Unit tests for LQR; produce Fig: cost vs time for optimal closed-loop; mini-checks (symmetry/PSD).
3. **Week 3:** Implement stochastic LQG Euler–Maruyama; Monte Carlo estimator; verify convergence in $(\Delta t, M)$.
4. **Week 4:** Implement analytic 2D LQ benchmark parameters; reproduce reference policy and cost numerically.

5. **Week 5:** Implement extended Hamiltonian H and partials for LQ case (closed form); add tests.
6. **Week 6:** Implement $\bar{u}_\theta$ (start with quadratic parameterization); verify learning recovers Riccati solution.
7. **Week 7:** Replace quadratic by richer family (splines or smooth NN); implement $\nabla \bar{u}_\theta$ and $\nabla^2 \bar{u}_\theta$ via autodiff.
8. **Week 8:** Implement discrete dual adjoint and pathwise gradient; validate with finite-difference gradient checks.
9. **Week 9:** Full training loop (SGD/Adam) on 2D LQ; produce learning curves; ablation on optimizer hyperparameters.
10. **Week 10:** Add smoothing H_δ interface and PSD-safe diffusion factorization; log clipping statistics.
11. **Week 11:** Implement switching/binary control benchmark (basket freeze); baselines always-continue / always-freeze.
12. **Week 12:** Train on switching benchmark; parameter sweeps in δ; report performance and stability.
13. **Week 13:** Consolidate experiments; generate thesis-ready plots/tables; start writing Methods + Experiments chapters.
14. **Week 14:** Write Introduction + Background; integrate benchmark results; add reproducibility appendix.
15. **Week 15:** Finalize thesis draft; internal review; fix missing explanations; ensure all figures regenerate.
16. **Week 16:** Final polish: code cleanup, tags/releases, final thesis PDF, and presentation slides.

11 References (minimal)

References

[1] W. H. Fleming and H. M. Soner, *Controlled Markov Processes and Viscosity Solutions*, Springer, 2nd ed., 2006.

[2] H. J. Kushner and P. G. Dupuis, *Numerical Methods for Stochastic Control Problems in Continuous Time*, Springer, 2nd ed., 2001.

[3] J. Yong and X. Y. Zhou, *Stochastic Controls: Hamiltonian Systems and HJB Equations*, Springer, 1999.

[4] R. Tempone, Controlled Diffusions as Unconstrained Smooth Optimization in a Potential: A Hamiltonian-Partials Reformulation with Discrete Dual Pathwise Gradients, Working draft, 2025.

A Appendix: quick checklist for every new model

- Specify (a, σ, h, g) and control set $\mathcal{A}$.
- Derive H and (if possible) closed forms for $\partial_\lambda H$, $\partial_V H$, $\partial_\gamma H$.
- If H is non-smooth, define a smooth H_δ and quantify the smoothing parameter you use.
- Add at least one unit test (even if small) for the new model.